

Sliding Window: The Core of Contiguous Subsegments

Mastering Dynamic Ranges, State Tracking, and Optimal Substrings

Presented by: QMML Society

Fundamental Concepts

The Sliding Window Technique

- **What is it?:** A subset of the Two Pointers technique forming a "window" over a contiguous sequence (array or string).
- **Fixed Window:** The window size k is constant. You move the whole window one element at a time.
- **Dynamic Window:** The window size changes. You expand the right pointer to find a valid window, and shrink the left pointer when the window becomes invalid.
- **Why use it?:** Reduces nested loops ($O(n^2)$) to a single linear pass ($O(n)$).

Visualization: Fixed (Static) Window

Scenario: Window of size $k = 3$. The window slides exactly *one element* at a time. Length never changes.

```
Array: [ 4, 2, 1, 7, 8, 1, 2, 8, 1, 0 ]  
      ^-----^  
      | k = 3 |
```

```
Step 1: [ 4, 2, 1 ] 7 8 1 2 8 1 0  
Step 2: 4 [ 2, 1, 7 ] 8 1 2 8 1 0  
Step 3: 4 2 [ 1, 7, 8 ] 1 2 8 1 0  
Step 4: 4 2 1 [ 7, 8, 1 ] 2 8 1 0
```

Notice how the "window frame" simply shifts right by 1 on every step.

Visualization: Dynamic Window

Scenario: Find the longest subarray with sum ≤ 10 . The window expands until invalid, then shrinks.

Array: [4, 2, 1, 7, 8, ...]

Step 1 (Expand):	[4]	Sum = 4 (Valid)
Step 2 (Expand):	[4, 2]	Sum = 6 (Valid)
Step 3 (Expand):	[4, 2, 1]	Sum = 7 (Valid)
Step 4 (Expand):	[4, 2, 1, 7]	Sum = 14 (INVALID! > 10)
Step 5 (Shrink):	4 [2, 1, 7]	Sum = 10 (Valid)
Step 6 (Expand):	4 [2, 1, 7, 8]	Sum = 18 (INVALID! > 10)
Step 7 (Shrink):	4 2 [1, 7, 8]	Sum = 16 (INVALID! > 10)
Step 8 (Shrink):	4 2 1 [7, 8]	Sum = 15 (INVALID! > 10)
Step 9 (Shrink):	4 2 1 7 [8]	Sum = 8 (Valid)

The **right** pointer aggressively expands. The **left** pointer defensively shrinks.

Essential Problem-Solving Patterns

Pattern	Logic	Example
Fixed Size	Initialise window of size k , slide by 1, updating state.	Maximum Average Subarray I
Dynamic (Longest)	Expand right. While invalid, shrink left. Update max length.	Longest Substring Without Repeats
Dynamic (Shortest)	Expand right. While valid, update min length and shrink left.	Minimum Window Substring
State Tracking	Use Map/Array to track frequencies/counts within the window.	Permutation in String

Optimization & Best Practices

- **Avoid Recomputation:** Don't recalculate the sum/state of the entire window every time it moves. Deduct the element leaving the window and add the element entering.
- **Character Frequencies:** For strings with typical ASCII characters, an `int[128]` or `int[26]` frequency array is drastically faster than a `HashMap`.
- **The `while` Loop:** When shrinking a dynamic window, use a `while` loop (not `if`) to remove elements from the left until the window is valid again.

Best Time to Buy and Sell Stock

Problem: Find the maximum profit from buying on one day and selling on a later day.

Sliding Window Logic:

- **Left** pointer (Buy), **Right** pointer (Sell).
- *If Buy > Sell:* Shift Buy to Sell (found a new lower baseline price).
- *Else:* Calculate profit and update max.

```
Prices: [ 7, 1, 5, 3, 6, 4 ]
```

```
Day 2: [7 (Buy)] [1 (Sell)]  
       7 > 1 -> Shift Buy to 1! (Found a better, cheaper deal)
```

```
Day 3: 7 [1 (Buy)] [5 (Sell)]  
       1 < 5 -> Profit is 4. Update Max to 4.
```

```
Day 4: 7 [1 (Buy)] 5 [3 (Sell)]  
       1 < 3 -> Profit is 2. (Max remains 4)
```

```
Day 5: 7 [1 (Buy)] 5 3 [6 (Sell)]  
       1 < 6 -> Profit is 5. Update Max to 5!
```

Language Deep Dive (Java vs. Python)

Java: String Iteration

- Avoid `s.substring()`, it creates a new String ($O(n)$).
- Use `s.charAt(i)` for fast $O(1)$ character access.
- `s.toCharArray()` is helpful but uses $O(n)$ extra memory.

Python: Dictionary & Strings

- `char in char_set` is $O(1)$.
- `Counter` from `collections` is great for frequency maps, but raw dictionaries or fixed arrays are faster.
- Slicing `s[left:right]` creates a copy, avoid it when just calculating lengths!

Brute Force Approach: Buy and Sell Stock

The logic: Calculate the profit for every possible pair of days.

Algorithm

1. Use **two nested loops**:
 - The outer loop **buy** selects the day to buy.
 - The inner loop **sell** (starting from **buy + 1**) selects the day to sell.
2. Calculate **profit = prices[sell] - prices[buy]**.
3. Track the maximum profit seen so far.

Complexity

- **Time:** $O(n^2)$ - We evaluate every possible pair. This will trigger a Time Limit Exceeded (TLE) on LeetCode!
- **Space:** $O(1)$ - Only tracking max profit variables.

Sliding Window Logic & Pseudocode

Instead of evaluating every future day for a given buy day, we can use Two Pointers!

- **left** tracks the absolute lowest price seen so far.
- **right** scans through the array day by day.

```
function maxProfit(array prices):
    left = 0 // Buy pointer
    right = 1 // Sell pointer
    max_profit = 0

    while right < length(prices):
        // Are we currently at a historically low buy price?
        if prices[left] < prices[right]:
            profit = prices[right] - prices[left]
            max_profit = maximum(max_profit, profit)
        else:
            // We found a new deeper valley!
            // Shift our buy pointer explicitly to this new low!
            left = right

        right = right + 1

    return max_profit
```

Optimal Buy and Sell Stock: Java

```
class Solution {
    public int maxProfit(int[] prices) {
        int left = 0; // Buy
        int right = 1; // Sell
        int maxProfit = 0;

        while (right < prices.length) {
            if (prices[left] < prices[right]) {
                int profit = prices[right] - prices[left];
                maxProfit = Math.max(maxProfit, profit);
            } else {
                left = right; // Found a new low!
            }
            right++;
        }
        return maxProfit;
    }
}
```

Optimal Buy and Sell Stock: Python

```
class Solution:
    def maxProfit(self, prices: list[int]) -> int:
        left = 0 # Buy
        right = 1 # Sell
        max_profit = 0

        while right < len(prices):
            if prices[left] < prices[right]:
                profit = prices[right] - prices[left]
                max_profit = max(max_profit, profit)
            else:
                left = right # Found a new low!

            right += 1

        return max_profit
```

Longest Substring Without Repeating Characters

Problem: Find the length of the longest substring without repeating characters.

✗ Brute-Force ($O(n^3)$)

Generate every possible substring boundary (i, j) , then scan each one with a HashSet.

```
s = "abcab"
```

```
[a] , [ab] , [abc] , [abca] (invalid) , [b] , [bc] , ...
```

Extremely slow. We repeatedly re-verify characters we already processed!

✓ Sliding Window ($O(n)$)

Expand **right** pointer. When a duplicate is hit, strictly shrink **left** to restore uniqueness!

String: a b c a b

Step 1: [a] b c a b → Set: {a}. Max: 1

Step 2: [a b] c a b → Set: {a,b}. Max: 2

Step 3: [a b c] a b → Set: {a,b,c}. Max: 3

Step 4: [a b c a] b → DUPLICATE 'a' ENCOUNTERED!

(Shrink left until the original 'a' is kicked out of the window)

a [b c a] b → Set: {b,c,a}. Max: 3

Step 5: a [b c a b] → DUPLICATE 'b' ENCOUNTERED!

a b [c a b] → Set: {c,a,b}. Max: 3

Brute Force Approach: Longest Substring

The logic: Generate all substrings and validate them.

Algorithm

1. Outer loop **i** marks the start of the substring.
2. Inner loop **j** marks the end of the substring.
3. For the sequence **s[i...j]**, iterate through it and use a standard **Set** to check if every single character is completely unique.
4. If valid, update **max_length = maximum(max_length, j - i + 1)**.

Complexity

- **Time:** $O(n^3)$ — Absolutely massive overhead. Evaluating n^2 substrings, and linearly scanning each sequence again takes $O(n)$.
- **Space:** $O(k)$ where k is the character set size.

Sliding Window Logic & Pseudocode

Instead of resetting upon a collision, we simply slide the "left" edge forward until the duplicate falls out of the window!

```
function lengthOfLongestSubstring(string s):
    left = 0
    char_set = empty Set
    max_length = 0

    for right from 0 to length(s) - 1:
        // While the new character is ALREADY in our window...
        while char_set.contains(s[right]):
            // ...shrink the window from the left to expel it!
            char_set.remove(s[left])
            left = left + 1

        // The window is valid and unique again
        char_set.add(s[right])
        max_length = maximum(max_length, right - left + 1)

    return max_length
```

Optimal Longest Substring: Java

```
class Solution {
    public int lengthOfLongestSubstring(String s) {
        Set<Character> charSet = new HashSet<>();
        int left = 0;
        int maxLen = 0;

        for (int right = 0; right < s.length(); right++) {
            // Shrink until valid
            while (charSet.contains(s.charAt(right))) {
                charSet.remove(s.charAt(left));
                left++;
            }
            charSet.add(s.charAt(right));
            maxLen = Math.max(maxLen, right - left + 1);
        }
        return maxLen;
    }
}
```


Optimal Longest Substring: Python

```
class Solution:
    def lengthOfLongestSubstring(self, s: str) -> int:
        char_set = set()
        left = 0
        max_len = 0

        for right in range(len(s)):
            # Shrink until valid
            while s[right] in char_set:
                char_set.remove(s[left])
                left += 1

            char_set.add(s[right])
            max_len = max(max_len, right - left + 1)

        return max_len
```

The Elegant Array Optimization

Goal: Avoid **HashSet** overhead by using a direct representation for ASCII characters and skip the **while** loop when possible.

Java Optimized:

```
public int lengthOfLongestSubstring(String s) {  
    Integer[] chars = new Integer[128];  
    int left = 0, res = 0;  
    for (int right = 0; right < s.length(); right++) {  
        char r = s.charAt(right);  
        if (chars[r] != null) {  
            left = Math.max(chars[r] + 1, left);  
        }  
        chars[r] = right;  
        res = Math.max(res, right - left + 1);  
    }  
    return res;  
}
```

Summary & Key Takeaways

- **Contiguous Elements:** If the problem asks for subarrays or substrings, sliding window is your best friend.
- **Two Pointers Foundation:** Sliding window intrinsically uses Left and Right pointers marking the boundaries.
- **Track Valid States:** The key to dynamic windows is defining what makes a window "valid" and adjusting Left to restore validity.
- **Optimization:** Swapping HashMaps for integer arrays (`int[128]`) yields massive performance gains for string problems.